

AI 前端开发提示词方法论（完整整合版）

适用平台：所有 AI 编程工具（Claude Code / Cursor / v0 / Bolt / Gemini 等） 核心原则：结构化优于即兴，数值化优于模糊，排除法比描述法更有效 参考来源：CSDN 前端提示词工程实战、掘金 AI 协作指南、腾讯云 AI 提示词整理、Addy Osmani 博客

一、六维度提示词模型

有效的前端开发提示词必须包含六个关键维度，缺少任何一个都可能导致 AI 生成的代码存在缺陷：

维度	作用	示例
目标场景	明确页面用途和核心价值	"开发一个电商商品详情页，用于展示商品信息和下单"
技术栈	限定使用的语言、框架和工具	"使用 HTML5 + Tailwind CSS + 原生 JavaScript，不依赖任何框架"
UI/UX 要求	描述视觉风格、布局和交互体验	"采用 Awwwards 级别的暗黑模式设计，布局需支持移动端适配"
功能细节	列出具体功能模块和交互逻辑	"包含商品图片轮播、数量加减、加入购物车动画"
代码规范	约定代码格式、注释和组织方式	"代码需按功能模块化，CSS 使用 BEM 命名规范"
输出格式	规定 AI 返回的内容结构	"先输出完整代码，再分模块解释关键实现"

二、CREATE 框架

字母	含义	说明	示例
C	Context 上下文	提供背景信息	"这是一个电商项目的购物车组件..."
R	Role 角色	设定 AI 角色	"你是一位 React 性能优化专家..."
E	Explicit 明确	明确任务目标	"请分析组件的重渲染问题..."
A	Audience 受众	指定目标读者	"解释需要面向初级开发者..."
T	Template 模板	提供输出模板	"请按以下格式输出：问题 -> 原因 -> 方案"
E	Examples 示例	提供参考示例	"参考以下代码风格..."

三、意图表达的三个层次

层次一（模糊意图）："帮我写个组件"

层次二（具体意图）："帮我写一个 React 登录表单组件"

层次三（精确意图）："使用 TypeScript 和 React Hook Form 编写一个登录表单组件，包含邮箱验证、密码强度检测，使用 Tailwind CSS 样式"

核心规律：每提升一个层次，AI 输出质量就显著提升。

四、Prompt 万能结构（可直接复用）

[项目约束]

技术栈: xxx / 目标环境: xxx / 禁止使用: xxx / 必须兼容: xxx

[设计规范]

风格: xxx / 色板: xxx / 字体: xxx / 布局模式: xxx / 质感效果: xxx

[交互要求]

- 页面过渡: xxx
- Hover 效果: xxx
- 加载状态: xxx
- 表单验证微交互: xxx

[架构标准]

- 响应式: xxx
- 无障碍: xxx
- 性能优化: xxx
- 组件化: xxx

[排除项]

避免: xxx / xxx / xxx

五、五大心法

1. **具体化**: 避免模糊描述, 明确每个需求细节
2. **完整性**: 要求端到端的完整建立, 而非基础机制
3. **多样性**: 鼓励给予多个设计方案进行比较
4. **专业性**: 使用行业标准术语和最佳实践
5. **排除法**: 明确说明要避免的模式和反模式

六、八大维度的正反例对照

1) 功能完整性

- 正例: Create a fully-featured implementation beyond the basics. Include as many relevant features and interactions as possible.
- 反例: Make a dashboard

2) 创意表现力

- 正例: Don't hold back. Give it your all. Create an impressive demonstration showcasing web development capabilities.
- 反例: Build a simple UI

3) 设计系统具体化

- 正例: 明确列出色板 (Dark blue and cyan) 、字体 (Inter for headings, system fonts for body) 、布局 (Card-based with subtle shadows) 、交互 (Hover states, transitions, micro-interactions) 、设计原则 (hierarchy, contrast, balance, movement)
- 反例: Make it look nice

4) 排除不良模式

- 正例: 明确列出要避免的设计 — Avoid: Generic centered layouts / Simplistic gradients / Uniform styling / Basic implementations
- 反例: Don't make it boring

5) 架构与构建标准

- 正例: 要求 Responsive design for all screen sizes / Accessibility compliance (WCAG 2.1 AA) / Performance optimizations / Clean, maintainable code structure
- 反例: Just make it work

6) 组件化思维

- 正例: Build using a component-based architecture with: Reusable UI components / Consistent design tokens / Modular CSS organization / Clear separation of concerns

7) 动态内容处理

- 正例: Implement real-time data visualization with: Live data updates every 30 seconds / Error handling for API failures / Loading states / Caching strategies

8) 用户体验流程

- 正例: Design the complete user journey including: Onboarding experience / Intuitive navigation patterns / Helpful error messages / Success confirmation

七、结构化提示词通用模板

```
## 角色
你是一位专业的【语言/框架】开发者，拥有【年限】年经验。

## 任务
请编写一个【功能描述】。

## 要求
1. 使用【技术栈】
2. 遵循【编码规范】
3. 包含【功能点1】【功能点2】【功能点3】
4. 添加【注释/测试/文档】

## 约束
- 【约束条件1】
- 【约束条件2】

## 参考风格
【示例代码】
```

八、关键洞察

- 1. **定义边界比定义内容更重要**：排除清单（Anti-Pattern）是控制 AI 输出的关键
- 2. **动效要系统化定义**：先定核心原则，再按类型分类描述
- 3. **视觉规范要数值化**：使用精确的色值、间距值、时间值
- 4. **分步迭代优于一步到位**：先核心功能再逐步添加
- 5. **规则文件是持久化的 Prompt**：将项目规范写入 CLAUDE.md / .cursorrules

九、参考来源

来源	核心贡献
CSDN - 前端提示词工程实战	六维度提示词模型
腾讯云 - AI 提示词最佳实践	全场景 Prompt 模板
掘金 - 前端开发者 AI 协作指南	CREATE 框架
Addy Osmani - LLM Coding Workflow	工作流与最佳实践